

Learning to Solve Hard Problems in RL for LLMs by Never Giving Up

Michael Noukhovitch*
Mila, Université de Montréal
Allen Institute for AI

Hamish Ivison†
University of Washington

Nathan Lambert†
Trillium Labs

Aaron Courville
Mila, Université de Montréal
Canada CIFAR AI Chair

Abstract

We demonstrate that training LLMs with RL does not improve performance equally across a dataset. RL shows large improvements on easy problems that an LLM is already good at solving, but small improvements on hard problems. We call this the *Matthew Effect* in RL for LLMs, after the phenomenon of cumulative advantage from economics and network science summarized as “the rich get richer”. The naive explanation is that hard problems require more compute to find a solution. We argue that modern RL methods are exacerbating the issue by wasting too much compute on easy problems and instead should dynamically reallocate how they use compute. We introduce *Never Give Up* (NGU), a simple adaptive sampling method that keeps generating samples for a problem until one is correct. By leveraging asynchronous RL, this naturally uses fewer samples to filter out easy problems and allocates more compute to solving harder problems. We investigate the design choices that affect NGU, such as off-policy robustness, and develop a set of best practices. On the math benchmark Deepscaler, NGU improves performance per compute, especially on harder problems. On a recent coding task, Manufactoria, standard GRPO with a per-test reward fails to fully solve problems that have a range of easy and difficult tests. NGU iteratively improves, solving harder and harder tests, until it learns to fully solve coding problems.

1 Introduction

Reinforcement learning (RL) is a standard method for post-training large language models (LLMs) in the modern AI pipeline (DeepSeek-AI et al., 2025b; Olmo et al., 2026). Pre-training and mid-training imbue LLMs with strong priors and instruction-following abilities, but post-training with RL enables models to improve beyond their supervised data (DeepSeek-AI et al., 2025a) in order to generalize to real world tasks (Chu et al., 2025).

Practitioners generally assume that by training with RL on a range of easy to difficult problems, our models will learn to solve problems across the entire distribution. We find that this isn’t true. In [Figure 1](#), we evaluate three different open-source RL-trained models from three different domains (math, coding, agentic coding) and find that easy problems receive a disproportionate amount of improvement compared to hard problems. Applying RL on LLMs shows a clear pattern of learning biased towards problems that the LLM was already good at. We connect this bias to a similar

*Correspondence to mnoukhov@gmail.com, see code at github.com/mnoukhov/never-give-up

†Work performed while at the Allen Institute for AI

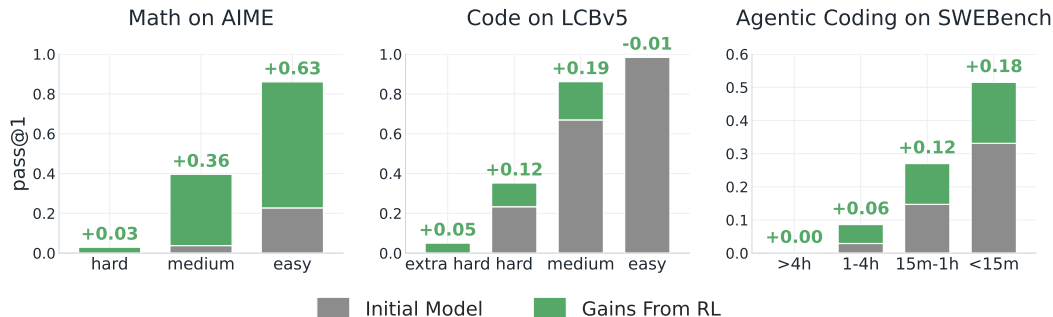


Figure 1: **The Matthew Effect in RL for LLMs.** We evaluate how RL improves performance using three different open-source RL-trained models from three domains: Olmo 3.1 RL-Zero Math on AIME, DeepCoder on LCBv5, and DeepSWE on SWEbenchVerified. We compare the RL-trained models to their initial models (Olmo 3 7B Base, Deepseek-Qwen-14B, Qwen3-32B), separating problems by difficulty. RL improves performance in proportion to the initial model’s performance: easy problems improve the most and hard problems improve the least.

phenomenon in network science and economics, the Matthew Effect (Merton, 1968), generally summarized as “the rich get richer”. This work proposes **the Matthew Effect in RL for LLMs**. In Section 2, we define and demonstrate how RL training improves on problems in proportion to how easily the initial LLM can already solve them. We then make three major efforts towards elucidating this issue and enabling RL to solve harder problems.

We find that inefficient compute allocation is one cause of the Matthew Effect in Section 3. Intuitively, harder problems require more samples to reach a solution. But we argue that modern RL for LLM methods exacerbate the issue by assigning equal compute to all problems, regardless of difficulty. We show how naively sampling more completions for every prompt can add noise from spurious failures on easy problems and argue for dynamic compute allocation.

We propose a dynamic sampling solution: Never Give Up (NGU) on unsolved prompts in Section 4. NGU iteratively samples completions to a prompt and either (1) stops if a correct answer is found or (2) puts the prompt back in the queue to continue sampling. To avoid infinitely sampling an impossible prompt, we continue with some probability p and otherwise give up. By leveraging modern asynchronous RL for LLMs (Noukhovitch et al., 2024), NGU uses fewer samples for easy problems and more samples for hard problems, naturally increasing their signal. We investigate important design choices for NGU and find that it helps to filter stale rollouts from the loss but including them in advantage calculations can improve performance.

We empirically validate NGU on math and code RL at larger scales in Section 5 and Section 6. We apply NGU to a larger scale math task, and find that it outperforms strong RL baselines in solving the hardest problems. We then demonstrate an adapted Matthew effect for large prompt harnesses on a code generation task. Where standard RL stagnates, unable to pass the hardest coding tests, NGU dynamically allocates more compute to solve the coding problems and pass all tests.

2 The Matthew Effect in RL for LLMs

To demonstrate the Matthew Effect, we analyze three open-source models trained exclusively with RL in specific domains: Olmo 3.1 RL-Zero on math (Olmo et al., 2026), DeepCoder on code completion (Luo et al., 2025b), and DeepSWE-Preview on agentic coding (Luo et al., 2025a). We group evaluation prompts by difficulty based on initial model performance: pass@1 on AIME (math), pass@1 on LCBv5 (code), and the human-time-based difficulty buckets provided in SWEbenchVerified. As shown in Figure 1, RL improvements scale with the model’s initial accuracy: tasks that are initially easier see larger gains, while harder tasks show more limited improvement. In other words, RL improves most where the model is already strong. This pattern holds consistently across domains, models (Olmo 3, DeepSeek-Qwen2.5-R1-Distill, and Qwen 3), and datasets. This is a form of *cumulative advantage*, and is analogous to the Matthew Effect in network science (Merton, 1968). Originally observed in scientific citation patterns—where well-known researchers receive

disproportionate credit for comparable work (Zuckerman, 1977)—we adapt the concept to RL for LLMs:

The Matthew Effect in RL for LLMs: RL improves performance on a task in proportion to a model’s initial competence—making easy tasks easier while hard tasks often remain difficult.

As this effect depends on the LLM’s initial conditions, we can see it as a sort of primacy bias in RL (Nikishin et al., 2022) where models are strongly impacted by their initial experiences. Previously, the RL primacy bias has focused on the issue of neural network plasticity loss when training from scratch (Nikishin et al., 2023). In contrast, the Matthew effect requires pre-existing biases and therefore occurs in pretrained models with strong priors.

3 Causes of the Matthew Effect

The Matthew Effect may be partially explained by the mechanics of modern approaches to RL for LLMs, specifically GRPO (Shao et al., 2024) and its variants. These methods use an empirical group baseline (Kool et al., 2019; Ahmadian et al., 2024): for each of the N prompts, we sample K completions and the advantage is computed as each completion’s reward minus the group mean, $A_i = r_i - \frac{1}{K} \sum_i r_i$. The consequence is straightforward — if all K completions fail and receive zero reward, the prompt contributes no gradient. This *signal loss* hypothesis (Xiong et al., 2025) is an intuitive explanation; as noted by prior work (Qu et al., 2026a), RL can fail to improve on difficult problems simply because correct solutions are never sampled. The standard remedy is to increase K , raising the probability of sampling at least one correct solution for harder prompts (Hu et al., 2025). To investigate whether signal loss is indeed the primary driver of the Matthew Effect, we design an RL testbed for mathematical reasoning using a small-scale but high-quality dataset.

Experimental Setup We finetune Qwen 2.5 0.5B Instruct (Qwen et al., 2025) on math problems from GSM8k (Cobbe et al., 2021). To avoid issues with inaccurate data, we train on the cleaned and verified subset, GSM8k Platinum (Vendrow et al., 2025). We separate our evaluation into 4 distinct levels of problem difficulty: easy, medium, hard, and extra hard. To generate our evaluation set, we sample 1024 completions for each prompt using the initial model and extract 8 samples that correspond to four levels of difficulty: pass@1 of 25% for easy, 10% for medium, 5% for hard, and 0% for extra-hard. Of extra-hard problems, 3/4 are completely unsolved in 1024 samples. We train our model with GRPO (Shao et al., 2024) sampling batches of N prompts and K completions-per-prompt. We leverage algorithmic refinements from recent works (Yu et al., 2025; Liu et al., 2025a) and train with off-policy asynchronous RL (Noukhovitch et al., 2024) as it is the standard in large-scale post-training (Cursor et al., 2026; GLM-5-Team et al., 2026). We filter out any prompt that receives no GRPO gradient (i.e., all-correct or all-incorrect completions) (Khatri et al., 2025). To maintain a constant batch size, we do active sampling (Olmo et al., 2026) i.e. if a prompt is filtered, we sample more prompts until we have a full batch with non-zero GRPO gradient³. We run all experiments for 3 seeds and report mean and standard deviation. See [Appendix D.1](#) for all experimental details.

Signal Loss? To test the signal loss hypothesis (Xiong et al., 2025), we examine the effect of increasing the number of completions-per-prompt K , intuitively reducing the chance of sampling all-incorrect responses. Specifically, we train with GRPO across four settings $K \in \{4, 8, 16, 32\}$, adjusting the number of prompts per batch $N \in \{64, 32, 16, 8\}$ to keep the total batch size $N \times K$ constant. As shown in [Figure 2](#), all settings exhibit the Matthew Effect: pass@1 improves more rapidly on easier problems than on harder ones. Counterintuitively, the smallest setting $K = 4$ performs best overall, which is clearly visible on the hardest problems (shown in red). This result suggests that signal loss is not the primary driver of the Matthew Effect. Holding total batch size and training steps fixed, the expected number of times each problem is sampled remains constant regardless of K . What K does control is the proportion of prompts filtered out due to a zero gradient — that is, groups where all completions are either entirely correct or entirely incorrect. Increasing K therefore does not increase the effective learning signal; it merely changes which prompts are discarded.

³This is the asynchronous analog to dynamic sampling used in synchronous RL (Yu et al., 2025)

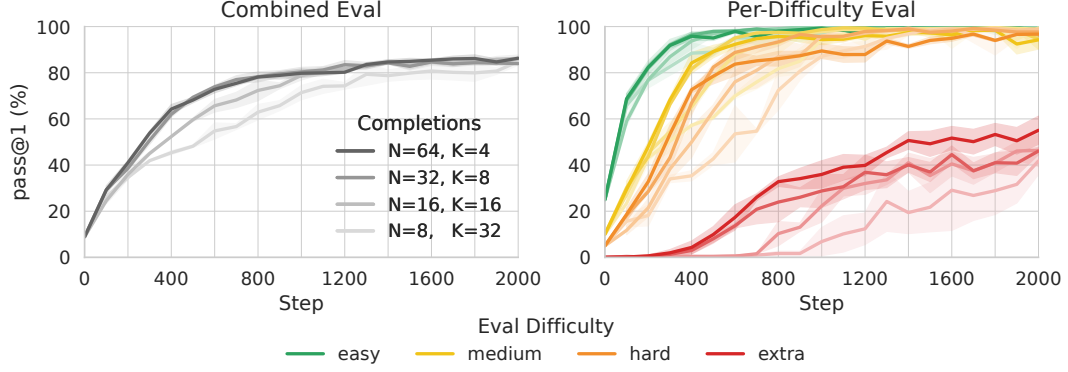


Figure 2: **More Completions-per-prompt K doesn’t necessarily solve harder problems.** Under a fixed total batch size, using more completions per prompt (higher K , lighter lines) underperforms using 4 completions per prompt ($K = 4$), even for harder problems.

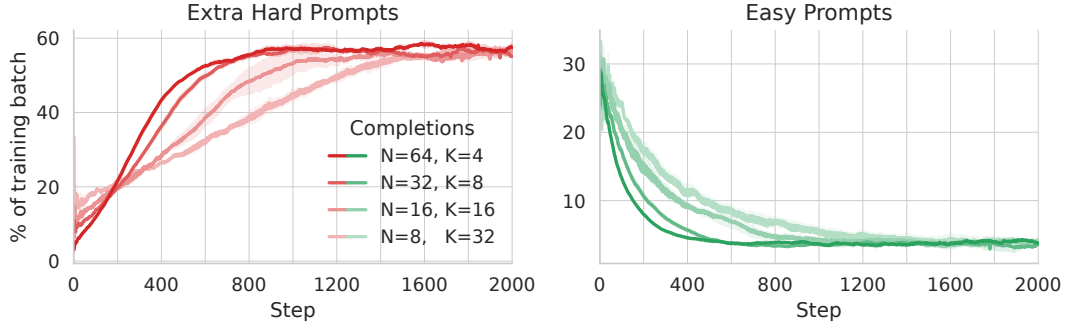


Figure 3: **Fewer Completions-per-prompt K can shift training batches towards harder prompts.** Using fewer completions per prompt (darker lines) results in more of the training batch comprising of hard training examples. This is due to more aggressive filtering of easy samples which enables focusing compute towards solving harder problems.

Increasing the number of sampled completions, K , has two competing effects. On one hand, larger K increases the probability of generating at least one correct completion for a difficult problem. On the other hand, it also increases the likelihood of sampling at least one *incorrect* completion for an otherwise easy problem. As a result, smaller values such as $K = 4$ require relatively little compute to filter out easy prompts and continue searching for harder ones. In contrast, larger values such as $K = 32$ will end up including many more easy prompts in the training batch because it only requires a single incorrect completion among the 32 samples to be included in training. Consequently, larger K values allocate a greater fraction of training updates to problems that are already largely solved. To illustrate this effect, we partition the training set into four equal difficulty quartiles and track the composition of training batches over time. As shown in Figure 3, early in training, $K = 4$ contains fewer hard prompts because the model initially struggles to solve them, whereas $K = 32$ includes substantially more. However, by step 200, the smaller- K setting has filtered out easy prompts much more aggressively, resulting in training batches that are increasingly concentrated on harder examples.

Signal Efficiency We propose an alternative interpretation: the *signal efficiency* hypothesis for RL on LLMs. The central issue is not merely whether enough completions are sampled for difficult problems but whether excessive compute is spent oversampling easy ones. From this perspective, RL training is fundamentally a problem of optimizing performance per unit of compute (Khatri et al., 2025). Standard GRPO always samples a fixed amount of completions K , which creates an inherent inefficiency: when K is too large, substantial compute is wasted generating redundant completions for already-solved prompts; when K is too small, difficult problems are undersampled and therefore contribute little useful learning signal. This perspective also helps explain the effectiveness of recent multi-stage RL curricula that begin training with smaller K values and later transition to larger ones

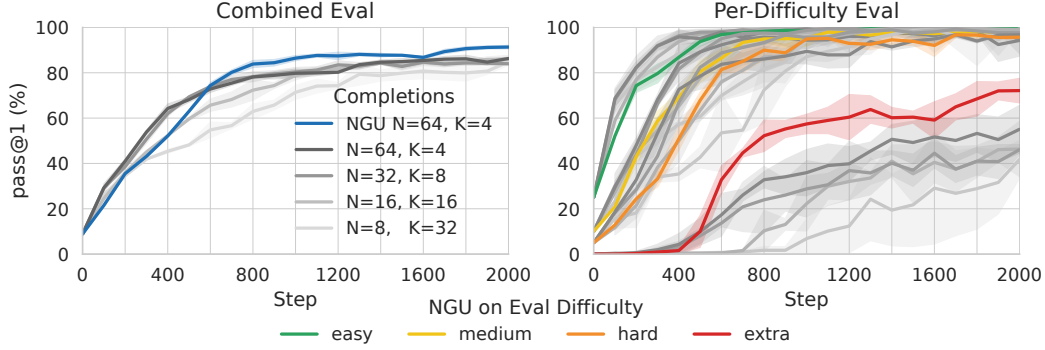


Figure 4: **Never Give Up improves on the hardest problems.** Using NGU improves overall pass@1 (blue line on left), especially improving performance on the hardest subset of samples (red line on right). Baseline GRPO results from Figure 2 are shaded in grey.

(Hu et al., 2025). Early low- K training can efficiently filter out easy prompts, while later high- K stages devote additional sampling budget to the increasingly difficult problems that remain unsolved. However, explicit curricula can be brittle and very sensitive to hyperparameters, we therefore aim to achieve signal efficiency using an adaptive, online method.

4 Countering the Matthew Effect with Never Give Up

Not Resampling Easy Prompts The simplest solution has been to exclude problems that are too easy from being resampled later in training (An et al., 2025). The issue is that this changes our training distribution to be harder and harder assuming that our model will never regress on previously-solved problems. In Appendix C.2 we find that our baseline training runs can perfectly solve a problem, but regress on it later in training when implementing this filtering strategy. So this approach can improve on the hardest problems at the expense of performance on easier problems.

Never Give Up Our goal is therefore to keep sampling from our full training distribution but more quickly filter easy prompts while reallocating compute to hard prompts. We propose *never give up*, a simple but effective way to leverage our asynchronous RL pipeline and reallocate compute to harder prompts. We first sample some small number of completions-per-prompt e.g. $K = 4$. If all our completions are correct, we can quickly filter this problem as too easy. If all our completions are wrong, the standard approach is to give up and sample another problem. Instead, we *never give up* (NGU) on solving this problem and sample K more completions, continuing until we solve the prompt. As certain problems may be too difficult, we actually continue sampling with probability p_{NGU} and give up on the problem with probability $1 - p_{\text{NGU}}$. This creates a geometric distribution of the total number of samples, allowing us to occasionally sample many completions on difficult prompts but keeping the average number of samples at $\frac{K}{1-p}$. See pseudocode for NGU in Appendix E.

We re-run our previous $N = 64$, $K = 4$ baseline but continue sampling with $p_{\text{NGU}} = 0.95$. As shown in Figure 4, NGU outperforms all previous baselines with its pass@1 gains coming specifically on the extra hard problem subset. Looking at what prompts NGU trains on in Figure 5, we see that NGU gets the best of both large K and small K . Early in training, NGU has as many hard prompts in the batch as $K = 32$ but later in training it has as many as $K = 4$, achieving close to the pareto-optimal amount of hard prompts in the training batch. Similarly, NGU achieves the best of both $K = 4$ and $K = 32$ for filtering easy prompts. Key to NGU’s success is an asynchronous RL infrastructure, so that quickly filtered easy prompts are replenished by harder prompts, therefore reallocating compute from easy to harder problems. We demonstrate the necessity of async RL for NGU and provide a detailed comparison to previous sync RL work (Xiong et al., 2025) in Appendix C.1

NGU: Maintaining Previous Completions Another advantage of NGU over plain GRPO with $K = 4$ is that we can get a richer feedback signal by collecting a larger group size for harder problems. Standard sampling with $K = 4$ may occasionally sample the right answer for a very difficult prompt but the advantage can be at most $1 - \frac{1}{4}$ in our GRPO setup, even if the many previous samples of this

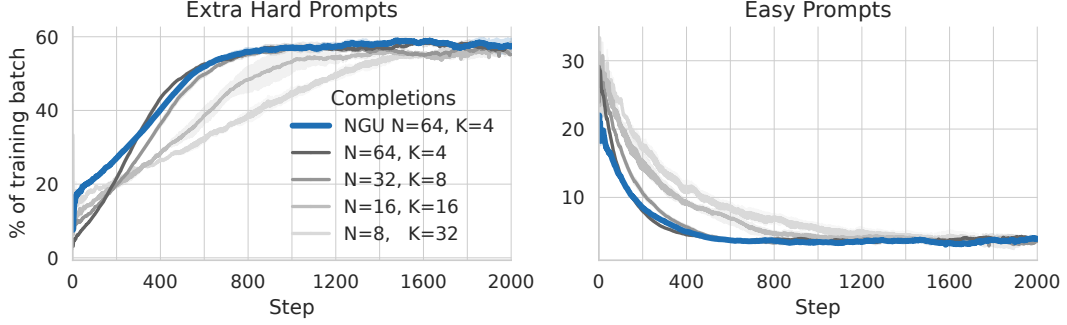


Figure 5: **Never Give Up achieves near pareto-optimality for training on hard prompts.** Using NGU results in the highest proportion of difficult samples and lowest proportion of easy samples across all training steps, suggesting it gets the benefits of both small and large K through its dynamic allocation. Baseline GRPO results shaded in grey (see Figure 3).

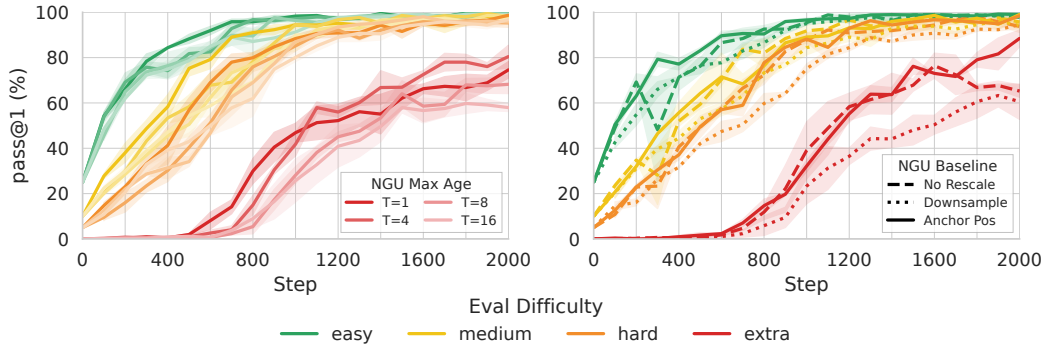


Figure 6: **Never Give Up requires filtering of stale/off-policy completions, but can use them for an improved GRPO baseline.** We measure performance across training and across samples of different difficulties. Using older completions ($T > 4$ steps off-policy) results in deteriorating performance (left), but these completions can be used for baselining. Using them for anchoring the positives outperforms other baselining approaches, especially downsampling (right).

prompt were unsuccessful. NGU resolves this problem by maintaining previous completions and, once a correct completion is found, forming one big GRPO group with all previous completions. A large group size with few correct answers greatly increases the advantage of the right answers, which allows for updating rare correct answers with a larger gradient. The disadvantage of this approach is that asynchronous NGU resampling can keep stale, off-policy negative completions from previous steps, which some recent work suggests can be harmful (Roux et al., 2025; Fu et al., 2025). To counter this, we keep track of the age of previous NGU completions and maintain only those that are less than T steps old. We run our experiment across $T = \{1, 4, 8, 16\}$ where $T = 1$ only maintains the current $K = 4$ completions. As shown in Figure 6, the performance increases from $T = 1 \rightarrow 4$ but then decreases with $T = 8, 16$.

NGU: Maintaining Counts Since it is optimal to only maintain previous completions with staleness of less than $T = 4$, it would be useful to still somehow leverage older completions that we filter from our training. An intuitive idea is to leverage all previous completions’ reward in calculating our true GRPO baseline $\bar{r}_{\text{NGU}} = \frac{1}{K * \text{NGU}} \sum_i r_i$, even if we don’t use all those completions in our update. The issue is that this will de-center the average advantage for a GRPO group as we will use $\bar{r}_{\text{NGU}}$ for a baseline but filter out some negative completions and their advantages for being too old.⁴ We propose an intuitive novel solution: maintain the advantage of the positive completions and evenly rescale the advantages of the negative completions to $\frac{n_+}{n_-} (1 - \bar{r})$ where $\frac{n_+}{n_-}$ is the ratio of positives to remaining negatives. This guarantees that, as before, the sum of advantages for our update group is 0. We call this *anchoring the positives* and compare it to two baselines. The first is simply to maintain a de-centered sum of advantages i.e. no rescaling. The second, proposed by

⁴Note that positives are new and only negatives can be stale, as we stop sampling once we achieve a positive

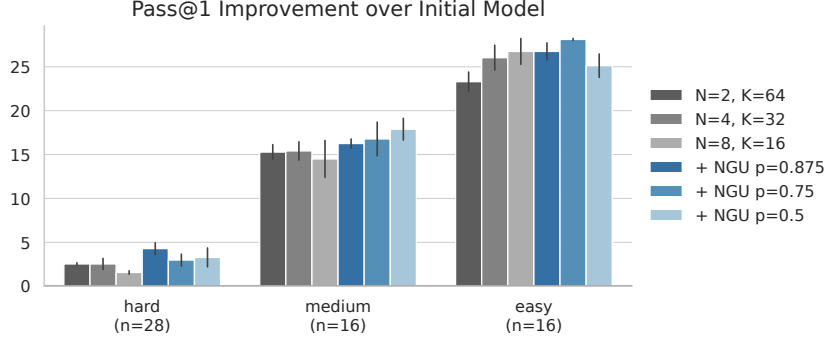


Figure 7: **NGU can improve the performance trade-off between hard/easy samples.** We train on Deepscaler with different values of K and eval on AIME and BRUMO 2025. NGU outperforms all options, especially on the hardest problems. Varying NGU p demonstrates a trade-off between improving on hard and easy evals, much like varying K . We show mean $\pm$ std dev across three seeds.

Xiong et al. (2025), is to downsample our negative completions so that there are an equal number of positive and negative completions then rescale advantages by $\frac{1}{\bar{r}_{\text{NGU}}}$ ⁵. We compare methods in Figure 6 and find that anchoring the positives is generally better, improving on the hardest prompts. In particular, downsampling and getting rid of useful negatives appears quite harmful to performance. In Appendix C.2, we also demonstrate that our NGU baseline with stale completions’ rewards is better than a standard GRPO baseline using only completions we train on.

5 Math: Scaling Up NGU

We now aim to validate our method and design choices at a larger scale, on a harder task.

Experimental Setup We train Qwen 3 4B-base (Yang et al., 2025) on a 10k subset of Deepscaler math (Luo et al., 2025c) following previous work (Li et al., 2025), and evaluate on difficult problems from recent math contests, AIME 2025 and BRUMO Nov 2025 (Dekoninck et al., 2026). We divide our combined evaluation prompts into difficulty buckets based on our initial model’s pass@64: hard includes all samples with pass@64=0, medium averages 2.9%, easy averages 46.1%.

We run our baselines varying completions-per-prompt $K \in \{16, 32, 64\}$ and maintain the same batch size with corresponding $N \in \{8, 4, 2\}$. We compare to $K = 16, N = 8$ with NGU varying $p_{\text{NGU}} \in \{0.5, 0.75, 0.875\}$ to give theoretical average sampling ranges of $K = 32, 64, 128$ respectively. For a fair comparison, we compute-match NGU by running all experiments for approximately 120 H100 hours regardless of the number of steps and run three seeds per setting.

NGU improves the easy/hard performance trade-off In Figure 7 we find that increasing K in our baseline trades off performance between easy and hard problems, similar to results on GSM8k. Varying p_{NGU} better trades off performance, improving more on hard problems without sacrificing as much performance on easy problems. As in GSM8k, we attribute NGU’s success to implicitly having more difficult prompts in the training batch, which we show in Figure 15 in Appendix C.3.

NGU outperforms curriculum learning We compare against a simple curriculum learning baseline that sets K per prompt using a model’s initial performance on that prompt. Fixed, larger K for hard prompts successfully reallocates compute and matches NGU’s performance on the hardest subset. But it degrades performance on easy problems, demonstrating that difficulty estimation is dynamic and can change over training. In Appendix C.3, we show results across eval datasets in Table 2 and results across difficulty levels in Table 3.

⁵This approach is reminiscent of MaxRL (Tajwar et al., 2026) which calculates GRPO advantage as $(r_i - \bar{r})/(\bar{r})$

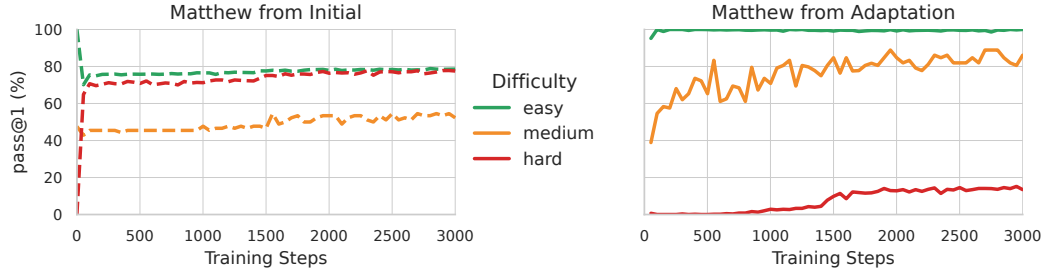


Figure 8: **The Harness-Aware Matthew Effect.** Performance on easy, medium, and hard problems over training. We can determine problem difficulty based on the model’s own pass rate either at the start of training (left) or after 100 steps when the model has adapted to the novel prompt/harness (right). Using the adapted difficulty ratings leads to a clear demonstration of the Matthew effect.

6 Coding: From Prompt Difficulty to Test Difficulty

Code generation, unlike math, can contain levels of difficulty not just between prompts but between tests within a single prompt. We demonstrate the benefit of NGU on a difficult coding task where standard RL doesn’t learn to pass the hardest coding tests.

Experimental Setup We experiment on a recent coding benchmark, ManuFactoria (PleasingFungus, 2010), a classic Flash game in which players build automated factories to sort robots based on their colored tape patterns. The underlying logic resembles constructing finite-state automata or tag systems and the benchmark allows for complex OOD training that evaluates RL’s ability to generalize. Following the original benchmark (Sun et al., 2025), we train Qwen3 4B Instruct (Yang et al., 2025) on a set of diverse and challenging coding problems, then evaluate on a test set that measures generalization. Each problem contains between 14 and 30 test cases, covering a range of difficulties. The final test cases in each example are frequently quite hard, making it feasible to solve a few test cases, but difficult to pass all tests. See all experimental details in [Appendix D.3](#).

The Harness-Aware Matthew Effect To evaluate the Matthew effect on ManuFactoria, we take 128 samples from our initial model, Qwen 3 4B Instruct, to assign a difficulty level for each prompt. We then train our model with GRPO following Sun et al. (2025), and plot the performance over each difficulty level over time. As shown in [Figure 8](#) (left), we see a surprising lack of a Matthew effect. This can be reasonably explained by ManuFactoria being both a novel task and using a long and novel prompt in order to make the output conform to requirements. The model is not well adapted to its prompt at initialization. Early RL steps are therefore more about adapting the model’s capabilities to the prompt, than a reflection of true improvement on the task (Liu et al., 2025a). Training shows two clear phases of (1) adapting to the task and prompt then (2) learning to solve the task with the specified prompt. For that reason, we propose to measure our Matthew effect using the model *after* prompt/harness adaptation has completed, around step 100. Re-classifying our problem difficulty after adaptation, in [Figure 8](#) (right), we again see a clear Matthew effect appear in our results. We therefore note the modified harness-aware Matthew effect for RL in LLMs:

The Harness-Aware Matthew Effect in RL for LLMs: RL improves an LLM’s performance on a task in proportion to the model’s *early* performance on the task *after accounting for adaptation to a prompt or harness*.

Signal Efficiency in Tests Sun et al. (2025) observed that a baseline GRPO run using per-test rewards was unable to fully solve the task. Although the model learned to solve roughly 80% of all coding tests, it rarely learned to pass all tests for any given problem. We interpret this phenomenon through the lens of the *signal efficiency* hypothesis. As shown in [Figure 8](#) (right), a standard RL run almost always solves the easiest tests, rarely exceeds 20% success on the hardest tests, and derives most of its reward variation from inconsistently solving medium-difficulty tests. Consequently, the majority of the training signal is allocated to repeatedly revisiting partially solved tests rather than driving progress on the hardest unresolved ones. In this regime, compute is spent inefficiently on

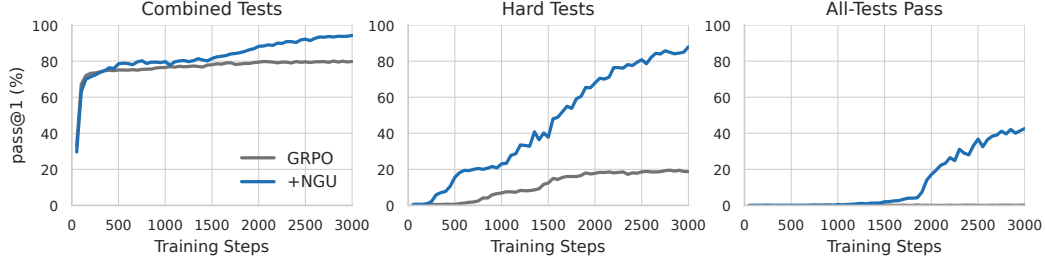


Figure 9: **Never Give Up enables solving the hardest tests on coding problems.** NGU enables continuously improving on all tests even as GRPO plateaus (left). The difference between the two methods is on hard tests (center). NGU improves on the hardest tests and learns to pass all of a problem’s tests, while GRPO remains unable to pass all tests on even a single example (right).

fluctuations around already-near-solved behaviors instead of generating an informative learning signal for full task completion. In contrast, NGU adaptively concentrates sampling on trajectories that improve over previous solutions. By continuing to sample until harder tests are solved, it allocates compute toward the hardest, most informative test cases rather than repeatedly oversampling easy or unstable intermediate cases.

We now run NGU on Manuformia with the same settings as the baseline, adding only $p_{\text{NGU}} = 0.95$, shown in Figure 9. As before, we confirm the main result of Sun et al. (2025) that standard GRPO cannot learn to solve all tests and find the reason is a stagnation in solving the hardest subset of tests (according to our harness-aware difficulty levels). In contrast, though NGU is initially slightly worse on the overall combined test pass-rate, it continues improving on hard tests and eventually transfers this capability into fully solving all tests and passing full problems.

Comparison to the Primacy Bias in RL The only method that Sun et al. (2025) found that could solve Manuformia was to first train with a reward *per-test* and, once performance has plateaued, restart training with a reward only if *all-tests* pass. The *all-tests* reward can avoid issues of signal efficiency but it isn’t possible to train with it from the start as the initial model never manages to pass all tests. To directly compare all-tests reward from Sun et al. (2025) with per-test reward using NGU we start both runs from the final GRPO baseline checkpoint after 3000 steps. In Figure 10 we compare both methods across compute time as all-tests reward runs slower per step due to larger amounts of filtering. We see that per-test reward with NGU approximately matches all-tests reward in compute efficiency. Interestingly, this result also demonstrates that RL for LLMs can train for 3000 steps with a suboptimal objective and then later recover when given a good training signal. In contrast, the primacy bias in RL (Nikishin et al., 2022) found that RL models trained from scratch will fail to recover from initial bad trajectories, due to plasticity loss. This result implies that RL for LLMs does not suffer the same plasticity loss and, despite being a form of primacy bias, the Matthew effect is distinct from plasticity.

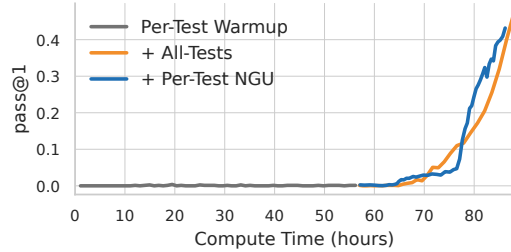


Figure 10: **Never Give Up can recover from a suboptimal model overfit to per-test reward.** NGU with per-test rewards performs similarly to standard RL with all-test pass reward.

7 Related Work

Many works in RL for LLMs have tackled solving harder problems and recent methods have usually focused on leveraging external information (Qu et al., 2026a; Wu et al., 2026), modifying the GRPO loss (Tajwar et al., 2026), or improving test-time search (Hubert et al., 2026). Another line of work has focused on creating an explicit curriculum of difficulty, either by removing easy problems

(An et al., 2025), curating an explicit dataset (Luo et al., 2025c), or adjusting the sampled data distribution (Shi et al., 2026; Qu et al., 2026b; Zheng et al., 2025). As NGU does not explicitly shift the data distribution but implicitly adapts it to improve performance-for-compute, these approaches are orthogonal and complementary to ours. The most similar method to NGU is Reinforce-Ada-Seq-Positive (Xiong et al., 2025), aimed at solving the issue of undersampling for hard prompts i.e. *signal loss*, as opposed to reallocating compute i.e. *signal efficiency*. This difference is driven by their use of synchronous RL, which removes any compute efficiency benefits of NGU. We delve into the differences and note the necessity of asynchronous RL to our method in [Appendix C.1](#). RL work prior to LLMs tackled highly difficult scenarios through the lens of exploration in sparse reward tasks (Bellemare et al., 2016). Methods generally focused on identifying promising states (Ecoffet et al., 2021) and using auxiliary losses to guide exploration (Badia et al., 2020). As our current RL for LLM setups are simple contextual bandits, not MDPs, we find NGU currently sufficient to improve performance in this setting. We provide an extended related work in [Appendix A](#).

8 Conclusion

In this work, we have highlighted *the Matthew Effect* in RL for LLMs, and shown that the standard approach results in disproportionately poor performance on the hardest problems. This also demonstrates how simple scalar values may not be sufficient for accurate evaluations of LLMs, and we should move towards dense evaluation signals. Our proposed solution, Never Give Up, represents a simple method for better allocating compute that allows substantial improvement on difficult tasks. Future work could examine more complex multi-step, agentic environments. As the feasibility of RL always comes down to a tradeoff of performance and compute (Khatri et al., 2025), future scaling of RL must continue to find ways to efficiently solve and progress on increasingly hard tasks.

Acknowledgements

MN thanks Shengyi Costa Huang for all the feedback, advice, and direction. MN thanks Samuel Lavoie, Dzmitry Bahdanau, and Finbarr Timbers for many helpful discussions and Yiyu Sun for help with implementing Manufactoria and reproducing results. MN was supported by the Fonds de recherche du Québec - Nature et Technologies. This research was supported by CIFAR, IVADO, and International Joint Research Project — KAIST NAIRL. We thank the Beaker and infrastructure teams at Ai2 for their support with Ai2 cluster. This material is based upon work supported by the National Science Foundation under Award No. 2413244 and research was enabled in part by support and compute provided by Mila (mila.quebec), Calcul Quebec (calculquebec.ca), and the Digital Research Alliance of Canada (alliancecan.ca).

References

- A. Ahmadian, C. Cremer, M. Gallé, M. Fadaee, J. Kreutzer, O. Pietquin, A. Üstün, and S. Hooker. Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs, Feb. 2024. URL <http://arxiv.org/abs/2402.14740>. arXiv:2402.14740 [cs].
- C. An, Z. Xie, X. Li, L. Li, J. Zhang, S. Gong, M. Zhong, J. Xu, X. Qiu, M. Wang, and L. Kong. POLARIS: A Post-Training Recipe for Scaling Reinforcement Learning on Advanced Reasoning Models, 2025. URL <https://hkunlp.github.io/blog/2025/Polaris>.
- J.-Y. Audibert, S. Bubeck, and R. Munos. Best Arm Identification in Multi-Armed Bandits. In *COLT*, 2021.
- A. P. Badia, P. Sprechmann, A. Vitvitskyi, D. Guo, B. Piot, S. Kapturowski, O. Tieleman, M. Arjovsky, A. Pritzel, A. Bolt, and C. Blundell. Never Give Up: Learning Directed Exploration Strategies, Feb. 2020. URL <http://arxiv.org/abs/2002.06038>. arXiv:2002.06038 [cs].
- M. G. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos. Unifying Count-Based Exploration and Intrinsic Motivation, Nov. 2016. URL <http://arxiv.org/abs/1606.01868>. arXiv:1606.01868 [cs].
- T. Chu, Y. Zhai, J. Yang, S. Tong, S. Xie, D. Schuurmans, Q. V. Le, S. Levine, and Y. Ma. SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-training, May 2025. URL <http://arxiv.org/abs/2501.17161>. arXiv:2501.17161 [cs].

- K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training Verifiers to Solve Math Word Problems, Nov. 2021. URL <http://arxiv.org/abs/2110.14168>. arXiv:2110.14168 [cs].
- Cursor, A. Chan, A. Shalaby, A. Wettig, A. Sanger, A. Zhai, A. Ajay, A. Nair, C. Snell, C. Lu, C. Shen, E. Jia, F. Cassano, H. Liu, H. Chen, H. Wildermuth, J. Jackson, J. Li, J. Katz, J. Yao, J. Hejna, J. Warner, J. Vering, K. Frans, L. Danilek, L. Wright, L. Cen, L. Melas-Kyriazi, M. Truell, M. de Jong, N. Jain, N. Schmidt, N. Wang, N. Muennighoff, O. Rybkin, P. Loh, P. Kravtsov, R. Yadav, S. Shah, S. Kottler, A. M. Rush, S. Zhang, S. Jain, S. Sankar, S. Heule, S. H. Sul, S. Asif, V. Rong, W. Zhu, W. Lin, Y. Wu, Y. Volkov, Y. Zemlyanskiy, Z. Holbrook, and Z. Zhang. Composer 2 Technical Report, Mar. 2026. URL <https://arxiv.org/abs/2603.24477v2>.
- DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *Nature*, 645(8081): 633–638, Sept. 2025a. ISSN 0028-0836, 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://arxiv.org/abs/2501.12948>. arXiv:2501.12948 [cs].
- DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan. DeepSeek-V3 Technical Report, Feb. 2025b. URL <http://arxiv.org/abs/2412.19437>. arXiv:2412.19437 [cs].
- J. Dekoninck, N. Jovanović, T. Gehringer, K. Rognvalddson, I. Petrov, C. Sun, and M. Vechev. Beyond Benchmarks: MathArena as an Evaluation Platform for Mathematics with LLMs, 2026. URL <https://arxiv.org/abs/2605.00674>. _eprint: 2605.00674.

- A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune. Go-Explore: a New Approach for Hard-Exploration Problems, Feb. 2021. URL <http://arxiv.org/abs/1901.10995>. arXiv:1901.10995 [cs].
- W. Fu, J. Gao, X. Shen, C. Zhu, Z. Mei, C. He, S. Xu, G. Wei, J. Mei, J. Wang, T. Yang, B. Yuan, and Y. Wu. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. In *Neural Information Processing Systems*. arXiv, 2025. doi: 10.48550/arXiv.2505.24298. URL <http://arxiv.org/abs/2505.24298>. arXiv:2505.24298 [cs].
- GLM-5-Team, A. Zeng, X. Lv, Z. Hou, Z. Du, Q. Zheng, B. Chen, D. Yin, C. Ge, C. Huang, C. Xie, C. Zhu, C. Yin, C. Wang, G. Pan, H. Zeng, H. Zhang, H. Wang, H. Chen, J. Zhang, J. Jiao, J. Guo, J. Wang, J. Du, J. Wu, K. Wang, L. Li, L. Fan, L. Zhong, M. Liu, M. Zhao, P. Du, Q. Dong, R. Lu, Shuang-Li, S. Cao, S. Liu, T. Jiang, X. Chen, X. Zhang, X. Huang, X. Dong, Y. Xu, Y. Wei, Y. An, Y. Niu, Y. Zhu, Y. Wen, Y. Cen, Y. Bai, Z. Qiao, Z. Wang, Z. Wang, Z. Zhu, Z. Liu, Z. Li, B. Wang, B. Wen, C. Huang, C. Cai, C. Yu, C. Li, C. Hu, C. Zhang, D. Zhang, D. Lin, D. Yang, D. Wang, D. Ai, E. Zhu, F. Yi, F. Chen, G. Wen, H. Sun, H. Zhao, H. Hu, H. Zhang, H. Liu, H. Zhang, H. Peng, H. Tai, H. Zhang, H. Liu, H. Wang, H. Yan, H. Ge, H. Liu, H. Chu, J. Zhao, J. Wang, J. Zhao, J. Ren, J. Wang, J. Zhang, J. Gui, J. Zhao, J. Li, J. An, J. Li, J. Yuan, J. Du, J. Liu, J. Zhi, J. Duan, K. Zhou, K. Wei, K. Wang, K. Luo, L. Zhang, L. Sha, L. Xu, L. Wu, L. Ding, L. Chen, M. Li, N. Lin, P. Ta, Q. Zou, R. Song, R. Yang, S. Tu, S. Yang, S. Wu, S. Zhang, S. Li, S. Li, S. Fan, W. Qin, W. Tian, W. Zhang, W. Yu, W. Liang, X. Kuang, X. Cheng, X. Li, X. Yan, X. Hu, X. Ling, X. Fan, X. Xia, X. Zhang, X. Zhang, X. Pan, X. Zou, X. Zhang, Y. Liu, Y. Wu, Y. Li, Y. Wang, Y. Zhu, Y. Tan, Y. Zhou, Y. Pan, Y. Zhang, Y. Su, Y. Geng, Y. Yan, Y. Tan, Y. Bi, Y. Shen, Y. Yang, Y. Li, Y. Liu, Y. Wang, Y. Li, Y. Wu, Y. Zhang, Y. Duan, Y. Zhang, Z. Liu, Z. Jiang, Z. Yan, Z. Zhang, Z. Wei, Z. Chen, Z. Feng, Z. Yao, Z. Chai, Z. Wang, Z. Zhang, B. Xu, M. Huang, H. Wang, J. Li, Y. Dong, and J. Tang. GLM-5: from Vibe Coding to Agentic Engineering, Feb. 2026. URL <http://arxiv.org/abs/2602.15763>. arXiv:2602.15763 [cs].
- B. He, Z. Qu, Z. Liu, Y. Chen, Y. Zuo, C. Qian, K. Zhang, W. Chen, C. Xiao, G. Cui, N. Ding, and Z. Liu. JustRL: Scaling a 1.5B LLM with a Simple RL Recipe, Dec. 2025. URL <http://arxiv.org/abs/2512.16649>. arXiv:2512.16649 [cs].
- J. Hu, M. Liu, X. Lu, F. Wu, Z. Harchaoui, S. Diao, Y. Choi, P. Molchanov, J. Yang, J. Kautz, and Y. Dong. BroRL: Scaling Reinforcement Learning via Broadened Exploration, Oct. 2025. URL <http://arxiv.org/abs/2510.01180>. arXiv:2510.01180 [cs].
- T. Hubert, R. Mehta, L. Sartran, M. Z. Horváth, G. Žužić, E. Wieser, A. Huang, J. Schrittwieser, Y. Schroecker, H. Masoom, O. Bertolli, T. Zahavy, A. Mandhane, J. Yung, I. Beloshapka, B. Ibarz, V. Veeriah, L. Yu, O. Nash, P. Lezeau, S. Mercuri, C. Sönne, B. Mehta, A. Davies, D. Zheng, F. Pedregosa, Y. Li, I. von Glehn, M. Rowland, S. Albanie, A. Velingker, S. Schmitt, E. Lockhart, E. Hughes, H. Michalewski, N. Sonnerat, D. Hassabis, P. Kohli, and D. Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, Mar. 2026. ISSN 1476-4687. doi: 10.1038/s41586-025-09833-y. URL <https://www.nature.com/articles/s41586-025-09833-y>.
- M. Jiang, E. Grefenstette, and T. Rocktäschel. Prioritized Level Replay. In *ICML*. arXiv, June 2021. doi: 10.48550/arXiv.2010.03934. URL <http://arxiv.org/abs/2010.03934>. arXiv:2010.03934 [cs.LG].
- D. Khatri, L. Madaan, R. Tiwari, R. Bansal, S. S. Duvvuri, M. Zaheer, I. S. Dhillon, D. Brandfonbrener, and R. Agarwal. The Art of Scaling Reinforcement Learning Compute for LLMs, Oct. 2025. URL <http://arxiv.org/abs/2510.13786>. arXiv:2510.13786 [cs].
- W. Kool, H. v. Hoof, and M. Welling. Buy 4 REINFORCE Samples, Get a Baseline for Free!, 2019. URL <https://openreview.net/forum?id=r1lgTGL5DE>.
- W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*. arXiv, Sept. 2023. doi: 10.48550/arXiv.2309.06180. URL <http://arxiv.org/abs/2309.06180>. arXiv:2309.06180 [cs].

- T. Li, Y. Zhang, P. Yu, S. Saha, D. Khashabi, J. Weston, J. Lanchantin, and T. Wang. Jointly Reinforcing Diversity and Quality in Language Model Generations, Sept. 2025. URL <http://arxiv.org/abs/2509.02534>. arXiv:2509.02534 [cs].
- Z. Liu, C. Chen, W. Li, P. Qi, T. Pang, C. Du, W. S. Lee, and M. Lin. Understanding R1-Zero-Like Training: A Critical Perspective. In *CoLM*. arXiv, Mar. 2025a. doi: 10.48550/arXiv.2503.20783. URL <http://arxiv.org/abs/2503.20783>. arXiv:2503.20783 [cs].
- Z. Liu, J. Liu, Y. He, W. Wang, J. Liu, L. Pan, X. Hu, S. Xiong, J. Huang, J. Hu, S. Huang, J. Obando-Ceron, S. Yang, J. Wang, W. Su, and B. Zheng. Part I: Tricks or Traps? A Deep Dive into RL for LLM Reasoning, Oct. 2025b. URL <http://arxiv.org/abs/2508.08221>. arXiv:2508.08221 [cs].
- M. Luo, N. Jain, J. Singh, S. Tan, A. Patel, Q. Wu, A. Ariyak, C. Cai, T. Venkat, S. Zhu, B. Athiwaratkun, M. Roongta, C. Zhang, L. E. Li, R. A. Popa, K. Sen, and I. Stoica. DeepSWE: Training a State-of-the-Art Coding Agent from Scratch by Scaling RL, 2025a. URL <https://pretty-radio-b75.notion.site/DeepSWE-Training-a-Fully-Open-sourced-State-of-the-Art-Coding-Agent-by-Scaling-RL-22281902c14>.
- M. Luo, S. Tan, R. Huang, A. Patel, A. Ariyak, Q. Wu, X. Shi, R. Xin, C. Cai, M. Weber, C. Zhang, L. E. Li, R. A. Popa, and I. Stoica. DeepCoder: A Fully Open-Source 14B Coder at O3-mini Level, 2025b. URL <https://pretty-radio-b75.notion.site/DeepCoder-A-Fully-Open-Source-14B-Coder-at-O3-mini-Level-1cf81902c14680b3bee5eb349a512a51>.
- M. Luo, S. Tan, J. Wong, X. Shi, W. Y. Tang, M. Roongta, C. Cai, J. Luo, T. Zhang, L. E. Li, R. A. Popa, and I. Stoica. DeepScaleR: Surpassing O1-Preview with a 1.5B Model by Scaling RL, 2025c. URL <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a>.
- R. K. Merton. The Matthew Effect in Science: The reward and communication systems of science are considered. *Science*, 159(3810):56–63, Jan. 1968. ISSN 0036-8075, 1095-9203. doi: 10.1126/science.159.3810.56. URL <https://www.science.org/doi/10.1126/science.159.3810.56>.
- E. Nikishin, M. Schwarzer, P. D’Oro, P.-L. Bacon, and A. Courville. The Primacy Bias in Deep Reinforcement Learning. In *Proceedings of the 39th International Conference on Machine Learning*, pages 16828–16847. PMLR, June 2022. URL <https://proceedings.mlr.press/v162/nikishin22a.html>.
- E. Nikishin, J. Oh, G. Ostrovski, C. Lyle, R. Pascanu, W. Dabney, and A. Barreto. Deep Reinforcement Learning with Plasticity Injection, May 2023. URL <http://arxiv.org/abs/2305.15555>. arXiv:2305.15555 [cs].
- M. Noukhovitch, S. Huang, S. Xhonneux, A. Hosseini, R. Agarwal, and A. Courville. Asynchronous RLHF: Faster and More Efficient Off-Policy RL for Language Models, Oct. 2024. URL <http://arxiv.org/abs/2410.18252>. arXiv:2410.18252.
- T. Olmo, A. Ettinger, A. Bertsch, B. Kuehl, D. Graham, D. Heineman, D. Groeneveld, F. Brahman, F. Timbers, H. Ivison, J. Morrison, J. Poznanski, K. Lo, L. Soldaini, M. Jordan, M. Chen, M. Noukhovitch, N. Lambert, P. Walsh, P. Dasigi, R. Berry, S. Malik, S. Shah, S. Geng, S. Arora, S. Gupta, T. Anderson, T. Xiao, T. Murray, T. Romero, V. Graf, A. Asai, A. Bhagia, A. Wetig, A. Liu, A. Rangapur, C. Anastasiades, C. Huang, D. Schwenk, H. Trivedi, I. Magnusson, J. Lochner, J. Liu, L. J. V. Miranda, M. Sap, M. Morgan, M. Schmitz, M. Guerquin, M. Wilson, R. Huff, R. L. Bras, R. Xin, R. Shao, S. Skjongsberg, S. Z. Shen, S. S. Li, T. Wilde, V. Pyatkin, W. Merrill, Y. Chang, Y. Gu, Z. Zeng, A. Sabharwal, L. Zettlemoyer, P. W. Koh, A. Farhadi, N. A. Smith, and H. Hajishirzi. Olmo 3, Apr. 2026. URL <http://arxiv.org/abs/2512.13961>. arXiv:2512.13961 [cs].
- OpenAI, C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. Józefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H. P. d. O. Pinto, J. Raiman, T. Salimans, J. Schlatter, J. Schneider, S. Sidor, I. Sutskever, J. Tang, F. Wolski, and S. Zhang. Dota 2 with Large Scale Deep Reinforcement Learning, Dec. 2019. URL <http://arxiv.org/abs/1912.06680>. arXiv:1912.06680.

- A. Piché, E. Kamaloo, R. Pardin, X. Chen, and D. Bahdanau. PipelineRL: Faster On-policy Reinforcement Learning for Long Sequence Generation, Sept. 2025. URL <http://arxiv.org/abs/2509.19128>. arXiv:2509.19128 [cs].
- PleasingFungus. Manufactoria. Browser game, 2010. URL <https://www.kongregate.com/games/pleasingfungus/manufactoria>. Accessed: 2026-09-02.
- Y. Qu, A. Setlur, V. Smith, R. Salakhutdinov, and A. Kumar. POPE: Learning to Reason on Hard Problems via Privileged On-Policy Exploration, Jan. 2026a. URL <http://arxiv.org/abs/2601.18779>. arXiv:2601.18779 [cs].
- Y. Qu, Q. Wang, Y. Mao, V. T. Hu, B. Ommer, and X. Ji. Can Prompt Difficulty be Online Predicted for Accelerating RL Finetuning of Reasoning Models? In *SIGKDD*. arXiv, Jan. 2026b. doi: 10.48550/arXiv.2507.04632. URL <http://arxiv.org/abs/2507.04632>. arXiv:2507.04632 [cs.AI].
- Qwen, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 Technical Report, Jan. 2025. URL <http://arxiv.org/abs/2412.15115>. arXiv:2412.15115 [cs].
- J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20*, pages 3505–3506, New York, NY, USA, Aug. 2020. Association for Computing Machinery. ISBN 978-1-4503-7998-4. doi: 10.1145/3394486.3406703. URL <https://doi.org/10.1145/3394486.3406703>.
- N. L. Roux, M. G. Bellemare, J. Lebensold, A. Bergeron, J. Greaves, A. Fr  chette, C. Pelletier, E. Thibodeau-Laufer, S. Toth, and S. Work. Tapered Off-Policy REINFORCE: Stable and efficient reinforcement learning for LLMs, Mar. 2025. URL <http://arxiv.org/abs/2503.14286>. arXiv:2503.14286 [cs].
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models, Apr. 2024. URL <http://arxiv.org/abs/2402.03300>. arXiv:2402.03300 [cs].
- T. Shi, Y. Wu, L. Song, T. Zhou, and J. Zhao. Efficient Reinforcement Finetuning via Adaptive Curriculum Learning. *TMLR*, June 2026. doi: 10.48550/arXiv.2504.05520. URL <http://arxiv.org/abs/2504.05520>. arXiv:2504.05520 [cs.LG].
- Y. Sun, Y. Cao, P. Huang, H. Bai, H. Hajishirzi, N. Dziri, and D. Song. RL Grokking Recipe: How Does RL Unlock and Transfer New Algorithms in LLMs?, Oct. 2025. URL <http://arxiv.org/abs/2509.21016>. arXiv:2509.21016 [cs].
- F. Tajwar, G. Zeng, Y. Zhou, Y. Song, D. Arora, Y. Jiang, J. Schneider, R. Salakhutdinov, H. Feng, and A. Zanette. Maximum Likelihood Reinforcement Learning. In *ICML*, Feb. 2026. URL <https://arxiv.org/abs/2602.02710v1>.
- J. Vendrow, E. Vendrow, S. Beery, and A. Madry. Do Large Language Model Benchmarks Test Reliability?, Feb. 2025. URL <http://arxiv.org/abs/2502.03461>. arXiv:2502.03461 [cs].
- Y. Wu, S. Li, Z. Wen, X. Zhou, A. Talwalkar, Y. Yang, W. Huang, and T. Cai. Learn Hard Problems During RL with Reference Guided Fine-tuning, Mar. 2026. URL <http://arxiv.org/abs/2603.01223>. arXiv:2603.01223 [cs].
- W. Xiong, C. Ye, B. Liao, H. Dong, X. Xu, C. Monz, J. Bian, N. Jiang, and T. Zhang. Reinforce-Ada: An Adaptive Sampling Framework under Non-linear RL Objectives, Dec. 2025. URL <http://arxiv.org/abs/2510.04996>. arXiv:2510.04996 [cs].
- A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li,

- M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. Qwen3 Technical Report, May 2025. URL <http://arxiv.org/abs/2505.09388>. arXiv:2505.09388 [cs].
- J. Yao, Y. Hao, H. Zhang, H. Dong, W. Xiong, N. Jiang, and T. Zhang. Optimizing Chain-of-Thought Reasoners via Gradient Variance Minimization in Rejection Sampling and RL. In *Neural Information Processing Systems*, Oct. 2025. URL [https://openreview.net/forum?id=tA0l08qS5N&referrer=%5Bthe%20profile%20of%20Tong%20Zhang%5D\(%2Fprofile%3Fid%3D~Tong_Zhang2\)](https://openreview.net/forum?id=tA0l08qS5N&referrer=%5Bthe%20profile%20of%20Tong%20Zhang%5D(%2Fprofile%3Fid%3D~Tong_Zhang2)).
- Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, X. Liu, H. Lin, Z. Lin, B. Ma, G. Sheng, Y. Tong, C. Zhang, M. Zhang, W. Zhang, H. Zhu, J. Zhu, J. Chen, J. Chen, C. Wang, H. Yu, Y. Song, X. Wei, H. Zhou, J. Liu, W.-Y. Ma, Y.-Q. Zhang, L. Yan, M. Qiao, Y. Wu, and M. Wang. DAPO: An Open-Source LLM Reinforcement Learning System at Scale, May 2025. URL <http://arxiv.org/abs/2503.14476>. arXiv:2503.14476 [cs].
- H. Zheng, Y. Zhou, B. R. Bartoldson, B. Kailkhura, F. Lai, J. Zhao, and B. Chen. Act Only When It Pays: Efficient Reinforcement Learning for LLM Reasoning via Selective Rollouts, June 2025. URL <http://arxiv.org/abs/2506.02177>. arXiv:2506.02177 [cs].
- H. Zuckerman. *Scientific elite: nobel laureates in the United States*. The Free Press [u.a.], New York, 1977. ISBN 978-0-02-935760-6.

A Extended Related Work

The problem of scaling RL performance for compute has been investigated across multiple different dimensions. Khatri et al. (2025) perform an extensive search across different algorithmic decisions. He et al. (2025) conduct a smaller scale study and focus on simple rules that improve performance for small models trained on Math. Recently Liu et al. (2025b) dive into the interactions between dataset composition and RL algorithms, noting best practices at their intersection. These papers all perform important investigations and we leverage several of their recommendations in our algorithmic choices.

Our specific goal of solving harder problems has long been a topic of interest in RL and our proposed method has similarities to many prior works. In bandits, Audibert et al. (2021) aim for best-arm selection and propose successive rejection as the basis for improved exploration. In deep RL, Jiang et al. (2021) aim to solve harder tasks and use TD-error to guide a curriculum towards harder levels of training. In recent RL for LLMs, GVM-RAFT (Yao et al., 2025) also proposes to vary group sizes as a form of curriculum, used in an EM method that leverages SFT to approximate RL. Their method relies on predicting the correct group size using gradient norms and also is not particularly effective when applied to GRPO. Another recent method, GRESO (Zheng et al., 2025), also seeks to improve performance for compute but focuses on avoiding signal loss, as in Xiong et al. (2025). They use an online curriculum, similar to No-Positive Resampling (An et al., 2025) but with a probabilistic filtering of problems, separately tracking too easy and too difficult problems. Compared to NGU, GRESO requires many more hyperparameters to define their curriculum and may not improve performance on the hardest examples as it explicitly samples fewer difficult problems to avoid all-zero-reward groups. Additionally, these previous works leveraging online curricula do not investigate code RL where a single problem can contain tests of variable difficulties.

B Limitations

The intuition behind Never Give Up is that our RL training can reallocate compute by quickly filtering easy problems. This implicitly assumes a range of difficulties for our RL training distribution with a sufficient amount of easy tasks that need filtering. As shown in Figure 16 in Appendix D.2, our Deepscaler task does indeed have a range of difficulties that leans towards difficult prompts, but has a reasonable amount of easy and medium prompts as well. If a task leans more heavily towards difficult problems, Never Give Up will likely not be effective. This is because NGU’s iterative sampling process may take longer to finish a whole group of completions compared to explicitly sampling larger K from the start. This means finished groups of NGU completions are generally more asynchronously off-policy and therefore provide a worse training signal (OpenAI et al., 2019), especially for negative samples (Roux et al., 2025). For this reason, we also do not recommend setting $K = 1$ and fully controlling batch size with p_{NGU} as it leads to more off-policy training.

C Extra Results

C.1 Comparison to Reinforce-Ada

Reinforce-Ada-Seq-Pos is the most similar method to Never Give Up in the current literature, but this work notably differs from Reinforce-Ada (Xiong et al., 2025) in both method and motivation.

Signal Loss vs Signal Efficiency Reinforce-Ada methods aim to solve a different issue, *signal loss*: when all completions to a prompt have the same reward, giving zero GRPO gradient. Signal loss also takes issue with a prompt giving all *positive* completions, and therefore no gradient. So other methods, such as Reinforce-Ada-Seq-Balance also aim to increase sampling on very easy problems in order to get an unlikely negative sample. In contrast, our proposed *signal efficiency* sees unlikely positive samples as a useful signal but unlikely negative samples as noise.

Reinforce-Ada-Seq-Pos vs Never Give Up Though both methods continue sampling until a positive completion is achieved, Reinforce-Ada-Seq-Pos differs empirically from NGU in three important ways:

Table 1: **Ablating the differences between NGU vs Reinforce-Ada-Seq-Pos on GSM8k.** Each design decision behind Never Give Up is shown to be crucial to performance: giving up with probability p , asynchronous RL, anchoring positives for rescaling reward baseline.

	easy	medium	hard	extra
Baseline GRPO	100.0 $\pm$ 0.0	99.9 $\pm$ 0.1	99.6 $\pm$ 0.2	47.1 $\pm$ 6.4
+ NGU $p = 1.0$	83.9 $\pm$ 0.7	61.2 $\pm$ 2.3	36.3 $\pm$ 3.3	2.7 $\pm$ 1.0
+ NGU $p = 0.75$ + sync RL	99.0 $\pm$ 0.5	97.3 $\pm$ 0.5	93.8 $\pm$ 2.3	14.8 $\pm$ 5.2
+ NGU $p = 0.75$ + async RL + downsampling	99.5 $\pm$ 0.3	96.7 $\pm$ 0.7	95.4 $\pm$ 0.9	63.9 $\pm$ 5.5
+ NGU $p = 0.75$ + async RL + Anchor Pos	100.0 $\pm$ 0.0	99.2 $\pm$ 0.4	98.6 $\pm$ 0.3	87.9 $\pm$ 2.2

1. No "giving up". Xiong et al. (2025) do not stop sampling until a positive is reached, equivalent to NGU with probability $p = 1$. Therefore, this method stalls on any prompt that is unsolvable by the model.
2. Synchronous RL. Xiong et al. (2025) propose continuing to sample in a synchronous RL framework. It therefore doesn't benefit from filtering positives quickly as the batch still waits for all (hard) completions to finish before sampling another prompt.
3. Downsampling completions. Xiong et al. (2025) generally strives for a balanced batch containing an equal number of positives and negatives. Therefore Reinforce-Ada-Seq-Pos discards negative completions so that they match the number of positive completions.

We ablate these three design decisions on the GSM8k task and show results in Table 1. We find that $p = 1.0$ leads to stalling and is empirically quite bad. NGU $p = 0.75$ with sync RL underperforms baseline GRPO, demonstrating the necessity of asynchronous infra for NGU to be effective. And finally, we find that downsampling is less effective than our choice of anchoring positives and, as shown in Figure 12, it is also worse than doing nothing.

C.2 GSM8k

In Figure 11, we show full overall and per-difficulty results when varying the maximum age of completions used in NGU.

In Figure 12, we show full overall and per-difficulty results for different ways of rescaling advantages in the presence of filtered negatives. We compare not rescaling, Xiong et al. (2025) downsampling, and our "anchoring the positives".

In Figure 13, we show full overall and per-difficulty results when varying the baselining method used in NGU.

In Figure 14, we run our baseline but shift our prompt distribution by not resampling any prompt that we solve in $16/16$ completions, following An et al. (2025). Even early in training we see that our easiest subset of prompts starts to degrade in performance and eventually all subsets are degrading other than the hardest subset (red). Our takeaway is that not resampling easy prompts can be detrimental to their performance over multiple epochs and we prefer to avoid explicitly and irreversibly changing our prompt distribution.

C.3 Deepscaler

Table Results In Table 2 we note the final pass@1 results for all methods on our eval dataset, AIME and BRUMO. Our GRPO baselines' performance is better than GRPO baselines from previous work (Li et al., 2025). Though we did not include it in the original paper, we note that we also ran $N = 16, K = 8$ and it underperforms all other baseline settings as it simply doesn't sample enough correct solutions on such a difficult dataset. This demonstrates how K that is too small can also be detrimental to performance. In Table 3, we show the exact values for Figure 7 i.e. improvement in pass@1 by difficulty subset.

Prompt-Difficulty Curriculum Baseline We also compare NGU to a simple baseline of curriculum learning, inspired by GVM-RAFT (Yao et al., 2025) and Reinforce-Ada-Est (Xiong et al., 2025).

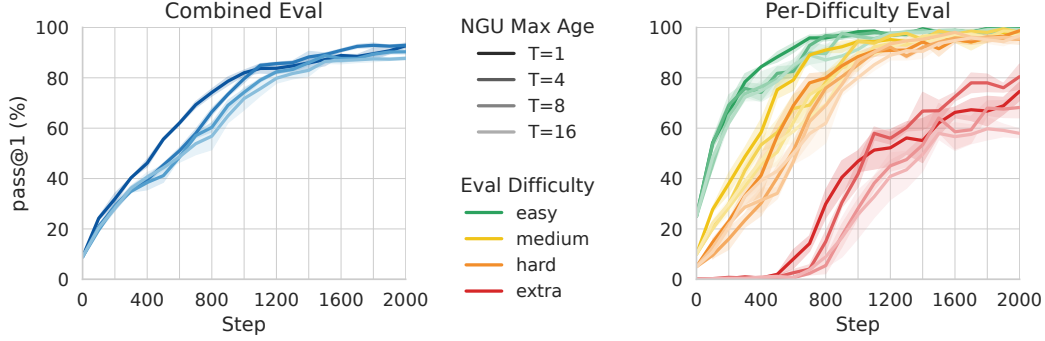


Figure 11: **Never Give Up must filter previous completions if they are too stale/off-policy.** Both overall (left) and per-difficulty (right), using more stale completions (lighter lines) lowers performance when using NGU.

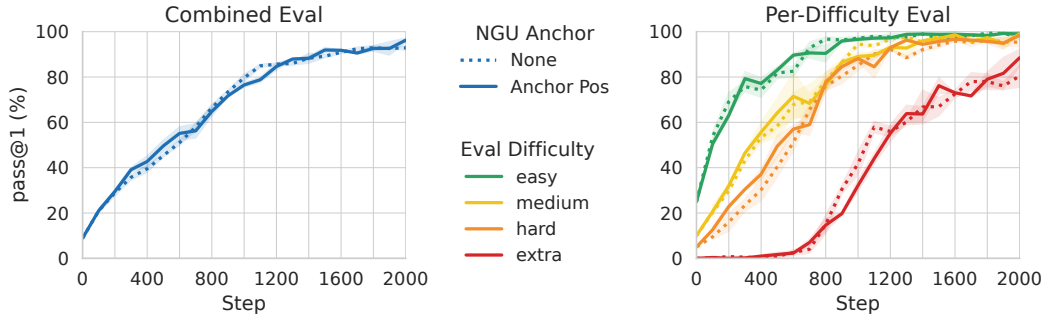


Figure 12: **Never Give Up can leverage filtered completions for an improved GRPO baseline.** We compare using stale completions with Anchor Pos rescaling to not using filtered completions for the GRPO baseline and find previous, filtered completions can be beneficial.

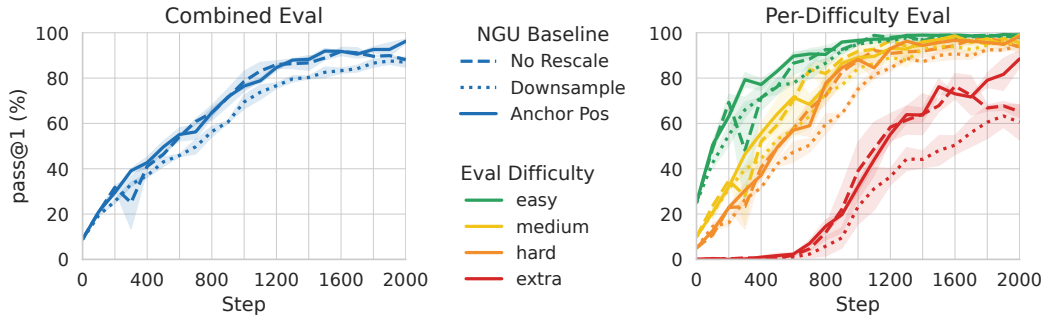


Figure 13: **Never Give Up can best leverage filtered completions for an improved GRPO baseline by rescaling rewards.** We ablate three methods for using stale completions. Downsampling underperforms both other approaches, and anchoring the positive completions performs best at the end of training.

We sample 32 completions per prompt with our initial model on our whole training dataset to get an estimate of each prompt’s difficulty. We then assign each prompt a number of completions K proportional to its expected pass-rate i.e. prompts with $\frac{8}{32}$ correct, or more, are assigned $K = 4$, $\frac{4}{32}$ are $K = 8$, $\frac{2}{32}$ are $K = 16$, and all less are $K = 32$. We sample exactly this K for each prompt and report results in Table 3. This difficulty curriculum greatly improves on the hardest subset, but it comes at the cost of large degradation on the easiest subset. This is in line with Section 4 experiments on the other curriculum learning method (An et al., 2025). Overall, it slightly underperforms the GRPO

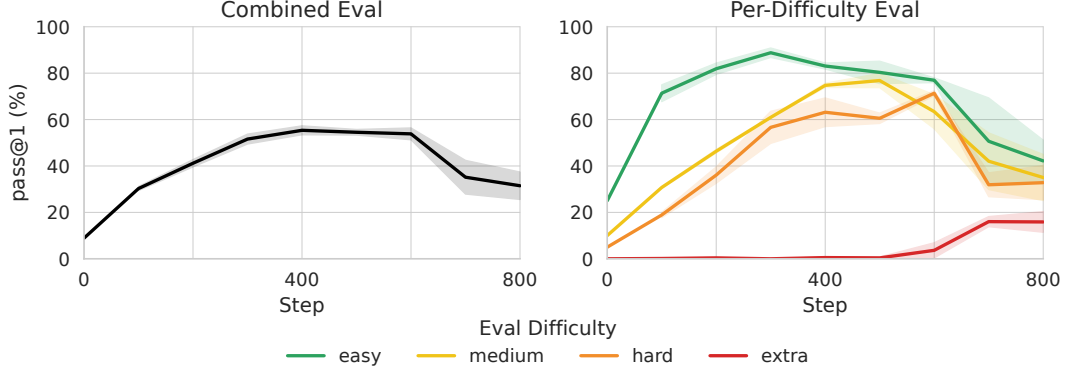


Figure 14: **GSM8k with No Resampling of Easy Prompts.** Following An et al. (2025), we don’t resample any prompts that achieve a perfect 16/16 correct completions during training. This fundamentally shifts the data distribution and we quickly see degradation on easy problems as our model shifts to answering exclusively harder prompts.

Table 2: **Deepscaler final pass@1 by dataset.** We show the pass@1 over three seeds for each setting of our GRPO baselines K and our NGU settings p_{NGU} . Results are mean $\pm$ std dev over three seeds. *Darling results are taken from the original paper which trained on a different 10k random subset.

	AIME 25	BRUMO 25	Average
Qwen 3 4B Base	7.63	16.3	12.6
N=2, K=64	20.3 $\pm$ 0.9	28.8 $\pm$ 0.7	24.5 $\pm$ 0.5
N=4, K=32	21.4 $\pm$ 1.3	29.2 $\pm$ 0.7	25.3 $\pm$ 0.5
N=8, K=16	20.0 $\pm$ 1.5	29.6 $\pm$ 1.4	24.8 $\pm$ 1.0
+ NGU $p=0.875$	21.8 $\pm$ 0.2	31.2 $\pm$ 1.3	26.5 $\pm$ 0.6
+ NGU $p=0.75$	21.3 $\pm$ 2.1	31.6 $\pm$ 1.5	26.4 $\pm$ 0.7
+ NGU $p=0.5$	22.0 $\pm$ 0.7	30.1 $\pm$ 1.6	26.1 $\pm$ 0.6
Darling (Li et al., 2025)*	20.1	31.7	25.9

baseline and implies that easy prompts may change in difficulty during training. We believe that RL should still sample a minimum amount per prompt, to maintain performance on easy problems.

Prompt Ratios In Figure 15, we plot the composition of our training batch. We divide training prompts into four equal groups of difficulty and plot percentage of our training batch, similar to Figure 3. As in GSM8k, we see that NGU shifts the difficulty of prompts that we train on by having the fewest easy prompts with non-zero gradient and the most hard prompts with non-zero gradient.

Table 3: **Deepscaler improvement in pass@1 by difficulty subset.** We plot the improvement in pass@1 over the initial Qwen 3 4B-base model after RL training, by difficulty splits. Results are mean $\pm$ std dev over three seeds.

	Easy	Medium	Hard	Total
N=2, K=64	23.3 $\pm$ 1.9	15.3 $\pm$ 1.5	2.5 $\pm$ 0.2	11.5 $\pm$ 0.5
N=4, K=32	26.0 $\pm$ 2.5	15.4 $\pm$ 1.8	2.5 $\pm$ 1.1	12.2 $\pm$ 0.5
N=8, K=16	26.8 $\pm$ 2.6	14.5 $\pm$ 3.6	1.6 $\pm$ 0.3	11.7 $\pm$ 1.0
+ NGU $p=0.875$	26.8 $\pm$ 1.7	16.3 $\pm$ 0.9	4.3 $\pm$ 1.2	13.5 $\pm$ 0.6
+ NGU $p=0.75$	28.1 $\pm$ 0.2	16.8 $\pm$ 3.3	3.0 $\pm$ 1.2	13.4 $\pm$ 0.7
+ NGU $p=0.5$	25.1 $\pm$ 2.3	17.9 $\pm$ 2.2	3.3 $\pm$ 1.9	13.0 $\pm$ 0.6
+ Sampling Curriculum	22.7 $\pm$ 1.6	15.5 $\pm$ 1.4	4.0 $\pm$ 1.1	12.1 $\pm$ 0.8

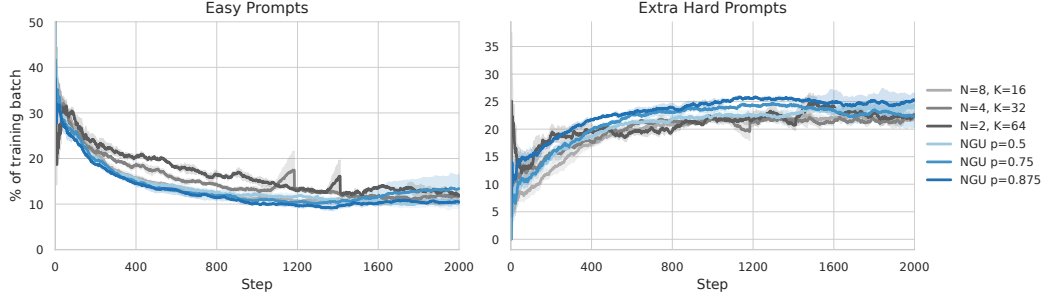


Figure 15: **NGU reallocates compute to more difficult prompts in Deepscaler.** As in GSM8k, Figure 3, we plot the ratios of the easiest and hardest prompts in our update batch over training. NGU increases the proportion of hard prompts and decreases the proportion of easy prompts. As before, NGU achieves the best of large K early in training and small K late in training.

D Experimental Details

For all experiments we use an asynchronous RL framework that leverages in-flight updates (Piché et al., 2025) to stay as on-policy as possible. We leverage DeepSpeed (Rasley et al., 2020) for training and vllm (Kwon et al., 2023) for inference.

D.1 GSM8k

We run GSM8k for 2000 steps using 4xL40s GPUs which takes approximately 10 hours. We note all important hyperparameters in Table 4

Table 4: **Hyperparameters for the Qwen2.5-0.5B-Instruct on GSM8K**

Hyperparameter	Value
KL beta	0.0
Learning rate	1×10^{-6}
Total Steps	2000
Temperature	1.0
Completions Per Prompt K	16
Prompts per Batch N	32
Advantage normalization	centered
Async steps	1
Prompt length	2048
Response length	4096
Clip higher	0.28
Non-stop penalty	False
Mask truncated completions	False

D.2 Deepscaler

We reproduce the Math RLVR setup of (Li et al., 2025) and achieve better baseline results. We take a random 10k subset of the Deepscaler dataset (Luo et al., 2025c). We check overlap of problems with our evals and find them to be minimal.

To evaluate the difficulty of our prompts and evals, we run the initial model using the Qwen 3 recommended temperature 0.7 and top-p 0.8 (Yang et al., 2025). When actually evaluating our RL-trained model, we find that the optimal temperature and top p are both simply 1. Our fastest run of 1000 steps uses 8xH100 GPUs for approximately 15 hours. We note all hyperparameters in Table 5. We show the difficulty of our Deepscaler 10k training dataset in Figure 16, including how the difficulty splits used for Figure 15.

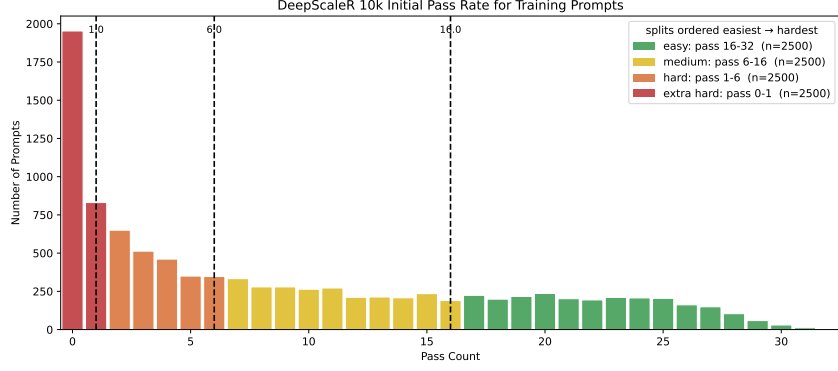


Figure 16: Difficulty of Deepscaler 10k dataset for Qwen 3 4B base. We take 32 samples on each prompt in the dataset and calculate the overall difficulty of the dataset. We find that a significant amount of data has $\text{pass}@32 = 0$ for the initial model. Dashed lines indicate borders between difficulty levels.

Table 5: Hyperparameters for Qwen3-4B-Base on Deepscaler Math

Hyperparameter	Value
KL beta	0.0
Learning rate	1×10^{-6}
Total Steps	1000
Temperature	1.0
Completions Per Prompt	16
Prompts per Batch	8
Advantage normalization	centered
Async steps	4
Prompt length	2048
Response length	8192
Clip higher	0.272
Non-stop penalty	False
Mask truncated completions	False

D.3 Manufactoria

We follow the original setup of Sun et al. (2025). We note that the true setup is actually slightly different from the details published in the Manufactoria paper and code, as discovered in communication with the original authors. We note all hyperparameters in Table 6. Each run of 3000 steps uses 2 nodes of 8xH100 GPUs for approximately 28 hours.

Table 6: Hyperparameters for Qwen3-4B-Instruct-2507 on Manufactoria

Hyperparameter	Value
KL beta	0.01
Learning rate	5×10^{-7}
Total Steps	1500
Temperature	1.0
Completions Per Prompt	16
Prompts per Batch	32
Advantage normalization	centered
Async steps	1
Prompt length	2048
Response length	12000
Clip higher	0.28
Non-stop penalty	False
Mask truncated completions	False

E Pseudocode

Algorithm 1 NEVER GIVE UP Sampling for Asynchronous RL Trainer

Require: Generator $\mathcal{G}$, Trainer $\mathcal{L}$, completions per prompt K

Require: continuation probability p_{NGU} , previous sample buffer $\mathcal{S}$, NGU age cutoff T

```

1: while trainer thread is running do
2:   Receive completions  $y_{1:K} \sim \pi_\theta(\cdot | x)$  from generator  $\mathcal{G}$ 
3:   Compute rewards  $r = \{r_i \leftarrow R(x, y_i) \text{ for } i = 1, \dots, K\}$ 
4:   Compute average reward  $\bar{r} = \frac{1}{K} \sum_{i \in 1 \dots K} r_i$ 
5:   Init count  $k = K$ 
6:   if  $\exists$  previous completions for this prompt  $y_{\text{NGU}} \in \mathcal{S}(x)$  then
7:     Pop previous completions  $y_{\text{NGU}}$ , rewards  $r_{\text{NGU}}$ , count  $k_{\text{NGU}}$ , baseline  $\bar{r}_{\text{NGU}} \leftarrow \mathcal{S}(x)$ 
8:     Update baseline  $\bar{r} \leftarrow \frac{\bar{r}_{\text{NGU}} * k_{\text{NGU}} + \bar{r} * K}{k_{\text{NGU}} + K}$ 
9:     Update count  $k += k_{\text{NGU}}$ 
10:    Extend with non-stale completions  $y \parallel= y_i \in y_{\text{NGU}}, \text{age}(y_i) \leq T$ 
11:    Extend with non-stale rewards  $r \parallel= r_i \in r_{\text{NGU}}, \text{age}(y_i) \leq T$ 
12:  end if
13:
14:  if  $r_i = 1 \forall_{r_i \in R}$  then                                 $\triangleright$  All completions correct; prompt is too easy
15:    Give Up on  $x$ 
16:
17:  else if  $\exists_{i \in [1, K]} \text{ s.t. } r_i > \bar{r}$  then                 $\triangleright$  Mixed outcomes provide GRPO gradient
18:    Pass  $(x, y, r, \bar{r})$  for GRPO update
19:
20:  else                                                         $\triangleright$  All completions wrong; prompt may be hard
21:    Sample  $u \sim \text{Uniform}(0, 1)$ 
22:    if  $u < p_{\text{NGU}}$  then                                       $\triangleright$  Never Give Up
23:      Add prompt  $x$  back to queue of generator  $\mathcal{G}$ 
24:      Add updated completions, baseline, count to buffer  $\mathcal{S}(x) \leftarrow y, k, \bar{r}$ 
25:    else
26:      Give Up on  $x$ 
27:    end if
28:  end if
29: end while

```
